AOI-Pipeline

Automated Optical Inspection for PCB Manufacturing Quality Control

---

Contents

1 Project Overview

This project provides a complete end-to-end system for detecting defects on printed circuit boards (PCBs): a YOLO-based component detector, an offline defect injector for generating synthetic training data, an accuracy evaluator, a standalone GUI labeler, a full training pipeline, and a feature-complete PySide6 GUI for live and offline inspection. Both a monolithic single-file GUI (`src/optical_inspection_system.py`) and a modular package version (`modular_aoi/`) are included.

The pipeline covers three main concerns:

Inspection
Given a "golden" (known-good) PCB image and a test board image, the system detects all components using a fine-tuned YOLOv8 model, aligns the test board to the golden reference via ORB feature matching and homography, then classifies each component slot as **OK**, **MISSING**, **MISALIGNED**, **WRONG COMPONENT**, or **WRONG POLARITY**.

Training data generation
Because real defective boards are scarce, `generate_defect.py` synthesises defective images by digitally injecting missing, wrong-component, and wrong-polarity defects into golden board photos. It produces lossless PNG outputs, annotated JPEG overlays, and a JSON ground-truth manifest.

Model training
The `train/` scripts implement a four-stage pipeline: offline augmentation → class-ID remapping → high-resolution tiling → two-phase frozen→full YOLO fine-tuning.

Supported Component Classes (25 total)

Connector (P), Resistor (R), Transformer (T), Diode (D), Capacitor (C), Transistor (Q), Jumper (J), Inductor (L), IC (U), Resistor Array (RA), Resistor Net (RN), Crystal (CR), IC (IC), Jumper (JP), Varistor (V), Button (BTN), Switch (SW), Switch (S), Test Point (TP), LED, Transistor (QA), Cap Array (CRA), Motor (M), Fuse (F), Ferrite Bead (FB).

2 Repository Structure

```
AOI-Pipeline/
+-- models/                          # Trained model weights
|   +-- best.pt
|
+-- data/                            # Training datasets and raw
    captured images
|   +-- Main_dataset/                # 23-image labelled dataset for
    training
|   |   +-- data.yaml
|   |   +-- images/                  # 23 PCB images (.jpg / .png)
|   |   \-- labels/                  # YOLO .txt annotations for each
    image
|   +-- Raw pics/                    # Original unlabelled source
    images (28)
|   |                                # includes 5 real Riyan DSLR boards
|   \-- Real_images/TEST/            # 7 real captured boards for
    validation
```

```
|       +-- data.yaml
|       +-- images/                # 7 Riyan DSLR board photos
|       \-- labels/                # (empty -- unlabelled test set)
|
+-- test_defects/                  # Pre-generated synthetic defect
    dataset
|   +-- original.jpg               # Golden board used to generate
    the set
|   +-- golden_components.json     # YOLO detections on the golden
    board
|   +-- manifest.json              # Ground-truth defect log for 20
    boards
|   \-- images/                    # board_000.png ... board_019.png
|
+-- src/
|   +-- pcb_aoi.py                 # Interactive CLI inspector
|   +-- generate_defect.py         # Synthetic defect generator
|   +-- aoi_evaluator.py           # Automated accuracy evaluator
|   +-- melss_labeler.py           # Standalone GUI labeler
|   +-- optical_inspection_system.py  # Full monolithic GUI (OIS v7.3)
|   \-- _ois_arrows/               # SVG arrow assets for the GUI
|
+-- train/
|   +-- augment_dataset.py         # Stage 1: offline augmentation
|   +-- tile_dataset.py            # Stage 3: high-res tiling
|   +-- train_finetune.py          # Stage 4: two-phase fine-tuning
|   \-- pipeline.py                # All 4 stages in one command
|
\-- modular_aoi/
    \-- ois/                       # Modular split of
       optical_inspection_system.py
        +-- __init__.py
        +-- aoi_engine.py
        +-- filters.py
        +-- main_window.py
        +-- theme.py
        +-- threads.py
        +-- utils.py
        +-- widgets.py
        \-- tabs/
            +-- aoi_tab.py
            +-- filters_tab.py
            +-- history_tab.py
            +-- infer_tab.py
            +-- run_tab.py
            \-- train_tab.py
```

3 Setup

3.1 Python Version

Python 3.9+ is recommended. PySide6 requires $\geq$ 3.8; PyInstaller builds target the same version.

3.2 Install Dependencies

```
pip install ultralytics opencv-python numpy PySide6 Pillow scipy \
            pyyaml platformdirs psutil pynvml pytesseract pyzbar sahi
```

Optional but recommended:

- `psutil` and `pynvml` — enable the real-time CPU/GPU status bar in the GUI.
- `pytesseract` — enables OCR filter nodes in the inspection pipeline.
- `pyzbar` — enables barcode/QR-code scanning filter nodes.
- `sahi` — enables tiled inference for very large PCB images (longest edge $>$ 1280 px in generator; $>$ 1920 px in OIS GUI).

3.3 Model Weights

All scripts expect a YOLOv8 model at `models/best.pt` relative to the repository root. Either train your own model using the `train/` scripts or place a pre-trained `.pt` file there.

4 File Reference

4.1 `src/`

`pcb_aoi.py`

Interactive CLI tool that compares test PCB boards against a golden reference board to detect **MISSING**, **MISALIGNED**, and **WRONG** components.

How it works: Runs YOLO detection on a golden board to build a reference component list, then for each test board: aligns it to the golden using ORB feature matching + RANSAC homography, runs YOLO detection on the aligned image, and compares detections using IoU matching and centre-distance thresholds.

Input: Prompts interactively for the YOLO model path, output directory, detection confidence, IoU match threshold, misalignment pixel threshold, and the image paths of the golden and test boards. No command-line flags — all input is entered at the prompts.

Output (written to the output directory):

- `golden_reference.json` — detected component list for the golden board
- `golden_annotated.jpg` — golden board with all detected components drawn
- `golden_raw.jpg` — copy of the raw golden image (used for alignment)
- `<board>_aligned.jpg` — test board after homography alignment
- `<board>_result.jpg` — annotated result image (colour-coded boxes)
- `<board>_report.json` — per-board JSON with full defect details
- `batch_summary.json` — summary across all tested boards

Usage:

```
python src/pcb_aoi.py
# Follow the interactive prompts to enter paths and thresholds
```

There are no command-line flags. The script reads defaults from constants at the top of the file (`DEFAULT_MODEL_PATH`, `DEFAULT_OUTPUT_DIR`, `DEFAULT_CONF`, etc.) and displays them as suggestions at each prompt.

generate_defect.py

Generates a dataset of synthetic defective PCB images by injecting controlled defects into a golden board photo. Supports three defect types:

- **missing** — colour-matched PCB-fill with cosine feathering
- **wrong_component** — cross-group component swap
- **wrong_polarity** — 180° rotation of polarized components

Default mix: missing 40%, wrong_component 35%, wrong_polarity 25%. Only polarized component types receive wrong_polarity defects: `IC (U)`, `IC (IC)`, `Transistor (Q)`, `Transistor (QA)`, `Diode (D)`, `LED`, `Capacitor (electrolytic)`.

Input: A golden board image (`--golden`), a YOLO `.pt` model (`--model`), and generation parameters.

Output (written to `--output-dir`):

- `images/*.png` — lossless defective board images
- `*_annotated.jpg` — overlay images showing injected defect locations
- `manifest.json` — ground-truth JSON mapping board names to defects
- `golden_components.json` — detected component list from the golden board
- `golden_annotated.jpg` — golden board annotated with all detections
- `summary.csv` — one row per board: name, defect count, types, labels

Usage:

```
# Generate 20 boards with up to 7 defects each
python src/generate_defect.py \
    --golden   /path/to/golden_board.jpg \
    --model    models/best.pt \
    --num      20 \
    --conf     0.15 \
    --defects-per-board 7 \
    --output-dir generated_defects \
    --seed     42

# Generate using SAHI tiled inference for large images (>1280px)
python src/generate_defect.py \
    --golden /path/to/golden_board.jpg \
    --model  models/best.pt \
    --num    20 \
    --use-sahi

# Compare a generator manifest against inspector output JSON
python src/generate_defect.py \
    --output-dir generated_defects \
    --compare    inspector_results.json
```

Flag	Default	Description
`--golden`	(hardcoded)	Path to the golden board image
`--model`	`models/best.pt`	Path to YOLO `.pt` model
`--num`	`20`	Number of defective boards to generate
`--conf`	`0.15`	YOLO detection confidence threshold
`--defects-per-board`	`7`	Number of defects to inject per board

Flag	Default	Description
`--output-dir`	`generated_defects`	Output directory
`--seed`	42	Random seed
`--use-sahi`	off	Use SAHI tiled inference for large boards
`--compare`	""	Path to inspector JSON to cross-reference against `manifest.json`

aoi_evaluator.py

Automated accuracy evaluator (v3.6). Injects defects into a golden board using the same injection logic as `generate_defect.py`, then immediately runs the AOI inspection algorithms on each synthetic board and reports precision, recall, F1, and a confusion matrix. No GUI. Designed for regression testing of the detection thresholds.

The evaluation pipeline is self-contained: it detects golden components, injects defects, runs the full multi-signal classification (SSIM, template matching, NCC polarity, patch variance), and records whether each injected defect was correctly identified.

Input: A golden board image and a YOLO model path.

Output: Printed table showing per-board injected vs. detected defects, a type confusion matrix (e.g. `WPOL` → `WCOM`, `WCOM` → `MISS`), and aggregate Precision / Recall / F1. Optionally saves an annotated debug image.

Usage:

```
# Basic 50-board evaluation
python src/aoi_evaluator.py \
    --golden /path/to/golden_board.jpg \
    --model  models/best.pt \
    --boards 50 \
    --defects 5

# 100-board run with verbose per-board output and debug signals
python src/aoi_evaluator.py \
    --golden  /path/to/golden_board.jpg \
    --model   models/best.pt \
    --boards  100 \
    --defects 7 \
    --conf    0.15 \
    --verbose \
    --debug

# Full trace and save annotated visualisation
python src/aoi_evaluator.py \
    --golden   /path/to/golden_board.jpg \
    --boards   20 \
    --defects  5 \
    --trace \
    --save_viz debug_viz.png
```

Flag	Default	Description
`--golden`	required	Path to golden board image
`--model`	`models/best.pt`	Path to YOLO `.pt` model

Flag	Default	Description
`--boards`	`50`	Number of synthetic test boards to evaluate
`--defects`	`5`	Max defects per board
`--conf`	`0.15`	YOLO detection confidence
`--seed`	`42`	RNG seed
`--verbose`	off	Print per-board injected vs. detected summary
`--debug`	off	Print SSIM/tmpl/var/path for failed slots (implies `--verbose`)
`--trace`	off	Print diagnostics for ALL Zone B/C slots on failed boards
`--save_viz`	`""`	Save annotated debug image (PNG) to this path

melss_labeler.py

Standalone PySide6 GUI for creating and editing YOLO-format bounding-box labels on PCB images. Supports auto-labeling using the YOLO model, manual box drawing, class selection, and project management.

Features:

- Draw bounding boxes by click-drag; edit or delete existing boxes
- Middle-mouse-button pan when zoomed; scroll-wheel zoom toward cursor
- Labels auto-saved on every add/edit/delete
- **Create Project** button: scaffolds `images/`, `labels/`, and `data.yaml` in a new directory
- Quick-select keyboard shortcuts for common classes: Transistor (Q), Resistor (R), Capacitor (C), IC
- Auto-label button: runs YOLO inference on all images in the project and pre-populates labels
- 25-class master list matching the training pipeline

Input: Launched interactively; user opens a project folder or creates one via the GUI.

Output: YOLO-format `.txt` label files written alongside images; a `data.yaml` generated automatically by the Create Project function.

Usage:

```
python src/melss_labeler.py
# Opens the GUI; use File -> Open Project to load an existing labels
    folder
```

No command-line arguments. The default model path is `models/best.pt` relative to the repository root.

optical_inspection_system.py

Monolithic PySide6 GUI for the complete MELSS Optical Inspection System (OIS v7.3). This is the single-file version of the application that is also split into `modular_aoi/` (see Section 4.3).

Tabs provided:

Run
: Live camera feed with real-time YOLO inference; capture golden reference, then inspect live boards with pixel-diff + component comparison; displays per-frame PASS/FAIL annunciator.

Golden / Train
: Manage the golden reference image, build labelled datasets from captured frames, configure the active filter pipeline for the reference.

Filters / Logic
Design and test the image pre-processing filter pipeline (CLAHE, Gaussian Blur, Sharpening, Gamma, Threshold, Bilateral, OCR, Barcode, ROI masking); auto-calibration via `AutoCalibrateWorker`.

AOI (Offline)
Batch inspect a folder of images against a golden reference without a live camera; results saved as JSON and annotated JPEGs.

Infer
Single-image or folder YOLO inference viewer.

History
SQLite-backed inspection history log with filterable table.

Input: Launched interactively. Model path and data directory are resolved at startup; the data directory uses `platformdirs` (`~/.ois_data` fallback).

Output: Inspection results saved to the data directory; history persisted in SQLite; annotated output images written per session.

Usage:

```
python src/optical_inspection_system.py
```

No command-line arguments.

src/_ois_arrows/

Contains four SVG files used as UI arrow assets by the OIS GUI: `arrow_up.svg`, `arrow_dn.svg`, `arrow_up_h.svg` (hover state), `arrow_dn_h.svg` (hover state). Referenced by `optical_inspection_system.py` at runtime and written to the data directory on first launch.

4.2 `train/`

augment_dataset.py

Stage 1 of the training pipeline. Takes a YOLO-format dataset (`images/` + `labels/`) and expands it to a target count using PCB-tailored augmentations with correct bounding-box remapping for every spatial transform.

Augmentation operations applied: horizontal flip, optional vertical flip, optional vertical flip, small rotation (±5°), 90°/180°/270° rotation, random scale (0.85–1.15×), random translation (±8%), mild perspective warp, brightness/contrast/saturation/hue/gamma jitter, Gaussian noise, motion blur, median blur, JPEG quality simulation, grid distortion.

Boxes smaller than 0.02% of image area or with less than 30% visibility after a spatial transform are dropped. After augmentation the images are shuffled and split 80/20 into `train/` and `val/` subdirectories, and a `data.yaml` is written.

Input: A source directory containing `images/` and `labels/` subdirectories with YOLO-format `.txt` label files.

Output: A new directory with `train/images/`, `train/labels/`, `val/images/`, `val/labels/`, and `data.yaml`.

Usage:

```
python train/augment_dataset.py \
    --src    /path/to/raw_project \
    --dst    /path/to/augmented_output \
    --target 280 \
    --seed   42
```

Flag	Default	Description
`--src`	(hardcoded)	Source folder with `images/` and `labels/`
`--dst`	(hardcoded)	Output augmented dataset folder
`--target`	280	Desired total image count (originals + generated)
`--seed`	42	Random seed

tile_dataset.py

Stage 3 of the training pipeline. Slices full-resolution PCB images into overlapping tiles so that small components (resistors, capacitors) are trained at their actual pixel size rather than being shrunk to near-invisible blobs.

Strategy: tiles are cut from the original full-resolution source images when available (best quality); falls back to the augmented set. Tile size defaults to 960 px with 25% overlap. Boxes that fall across tile boundaries are clipped; boxes with less than 25% visibility or less than 16 px^2 area after clipping are dropped. Empty (background-only) tiles are kept at a configurable rate (default 5%).

Input: A source image directory (`--src`) and optionally a separate augmented dataset directory (`--aug`) used to inherit class names and the validation split.

Output: A tiled dataset directory with `train/images/`, `train/labels/`, `val/images/`, `val/labels/`, and `data.yaml`.

Usage:

```
# Tile from original source images; use augmented set for val split
python train/tile_dataset.py \
    --src  /path/to/original_project \
    --aug  /path/to/augmented_output \
    --dst  /path/to/tiled_output \
    --tile    960 \
    --overlap 0.25 \
    --bg-keep 0.05

# Tile from the augmented set directly
python train/tile_dataset.py \
    --src /path/to/augmented_output \
    --dst /path/to/tiled_output
```

Flag	Default	Description
`--src`	required	Source dataset directory (originals preferred)
`--aug`	None	Augmented set dir (for class names and val split)
`--dst`	required	Output tiled dataset directory
`--tile`	960	Tile size in pixels
`--overlap`	0.25	Tile overlap fraction
`--bg-keep`	0.05	Fraction of empty tiles to keep
`--min-vis`	0.25	Minimum box visibility after tile clipping
`--min-area`	16	Minimum box area (px^2) after clipping
`--no-aug-val`	off	Also tile the val set (default: copy val untiled from `--aug`)
`--seed`	42	Random seed

train_finetune.py

Stage 4 of the training pipeline. Fine-tunes an existing YOLOv8 `.pt` model on a tiled PCB dataset using a two-phase training strategy optimised for small high-resolution PCB datasets.

Phase 1 — Frozen backbone warm-up (10 epochs by default). Trains only the detection head with slightly elevated learning rate. Early stopping disabled.

Phase 2 — Full fine-tune with all layers unfrozen. AdamW optimiser, cosine LR schedule, mixed precision (AMP), early stopping with patience=20. Exports the best weights to ONNX (opset 17, simplified) and TorchScript after training.

Key hyperparameters (v4): `lr0=0.0005`, `lrf=0.05`, `momentum=0.937`, `weight_decay=0.0005`, `mosaic=0.4`, `iou=0.60`, `scale=0.20`, `erasing=0.20`.

Input: A `data.yaml` from the tiled dataset, a base `.pt` model, and training parameters.

Output: Saved to `PROJECT_DIR/<name>/`: YOLO training artefacts, `weights/best.pt`, `weights/last.pt`, `best.onnx`, `best.torchscript`.

Usage:

```
python train/train_finetune.py \
    --data   /path/to/tiled_output/data.yaml \
    --model  models/best.pt \
    --name   my_run_v2 \
    --epochs 120 \
    --imgsz  960 \
    --batch  4 \
    --device 0
```

Flag	Default	Description
`--data`	(hardcoded)	Path to `data.yaml`
`--model`	`models/best.pt`	Base `.pt` to warm-start from
`--name`	`gatekeeper_v4_tiled`	YOLO run name
`--epochs`	`120`	Total epochs across both phases
`--imgsz`	`960`	Training image size
`--batch`	`4`	Batch size (`-1` for auto)
`--device`	`0`	Device: `0` (GPU 0), `cpu`, or `0,1` for multi-GPU

pipeline.py

Orchestrates all four training stages end-to-end in a single command: augmentation → class-ID remapping → tiling → two-phase fine-tuning. Individual stages can be skipped or the run can be started from a specific stage.

Stage 2 (Remap) is performed internally by `pipeline.py` and not exposed as a standalone script: it scans the augmented dataset's label files to find which class IDs are actually used, remaps them to a compact sequential range, and rewrites both label files and `data.yaml`.

Input: A raw labelled dataset directory and a base YOLO model.

Output: An augmented dataset, a tiled dataset, and final trained weights — all at configured output paths.

Usage:

```
# Full run (all 4 stages)
python train/pipeline.py \
    --src      /path/to/raw_project \
    --dst-aug  /path/to/augmented_output \
    --dst-tile /path/to/tiled_output \
    --model    models/best.pt \
    --name     my_run_v2 \
    --epochs   120 \
    --imgsz    960 \
    --batch    4 \
    --device   0

# Skip augmentation and remapping (dataset already augmented)
python train/pipeline.py \
    --src      /path/to/raw_project \
    --dst-aug  /path/to/augmented_output \
    --dst-tile /path/to/tiled_output \
    --skip-augment --skip-remap

# Start from the tiling stage
python train/pipeline.py \
    --dst-aug  /path/to/augmented_output \
    --dst-tile /path/to/tiled_output \
    --from-stage tile

# Dry-run the class remap (print plan, do not write files)
python train/pipeline.py \
    --dst-aug /path/to/augmented_output \
    --from-stage remap --dry-run-remap
```

Flag	Default	Description
`--src`	(hardcoded)	Raw images + labels source directory
`--dst-aug`	(hardcoded)	Augmentation output directory
`--dst-tile`	(hardcoded)	Tiling output directory
`--model`	`models/best.pt`	Base `.pt` model
`--name`	`gatekeeper_v4_tiled`	YOLO run name
`--from-stage`	—	Start from: `augment`, `remap`, `tile`, or `train`
`--skip-augment`	off	Skip Stage 1
`--skip-remap`	off	Skip Stage 2
`--skip-tile`	off	Skip Stage 3
`--skip-train`	off	Skip Stage 4
`--dry-run-remap`	off	Print remap plan without writing files
`--target`	280	Augmentation target count
`--tile`	960	Tile size (px)
`--overlap`	0.25	Tile overlap fraction
`--epochs`	120	Training epochs
`--imgsz`	960	Training image size
`--batch`	4	Batch size
`--device`	0	Training device

4.3 `modular_aoi/ois/`

This package is a modular refactoring of `src/optical_inspection_system.py`. All functionality is preserved; the code is split into focused modules. The entry point is

`modular_aoi/ois/main_window.py`.

`utils.py`

Pure data layer — no Qt widgets. Contains: optional-dependency feature flags (`HAS_CV2`, `HAS_NP`, `HAS_YOLO`, `HAS_SAHI`, `HAS_OCR`, `HAS_ZBAR`), the portable data-directory resolver (`DATA_ROOT` via `platformdirs` or `~/.ois_data`), the `_AOIComp` dataclass, SAHI inference constants, `ProjectConfig`, `SettingsDialog`, `HistoryDB` (SQLite history store with WAL journal mode), `MASTER_CLASS_LIST` (the 25-class list), `match_detections`, and helper functions (`calc_iou`, `safe_predict`, `load_optimized_yolo`, `find_best_pt`, `build_golden_dataset`, `DEFAULT_AUG`).

Not a runnable script; imported by all other modules in the package.

`filters.py`

Filter node system and auto-calibration. Defines the `FilterNode` base class and all concrete filter implementations: `CLAHEFilter`, `GaussBlurFilter`, `SharpenFilter`, `GammaFilter`, `ThresholdFilter`, `BilateralFilter`, `OCRFilter`, `BarcodeFilter`, `ROIFilter`. Provides `apply_filters(pipeline, image)` and `run_roi(roi_filter, image)`. Also contains `AutoCalibrateWorker` (a `QThread` that scores filter pipeline variants against a reference image) and the `auto_build_pipeline` function.

Not a runnable script; imported by `aoi_engine.py`, `threads.py`, and several tabs.

`aoi_engine.py`

Core AOI inspection algorithms. Contains all detection and comparison logic: `_aoi_bbox_overlap`, `_aoi_nms_by_centre`, `_aoi_infer_arr` (YOLO/SAHI inference returning `_AOIComp` lists), `_aoi_hungarian_match`, `_aoi_region_diff`, `_aoi_calibrate`, `_aoi_ncc_polarity` (NCC-based polarity flip detection), `_aoi_patch_ssim`, `_aoi_patch_variance`, `_aoi_patch_tmpl`, `_aoi_same_det_is_local`, `_aoi_check_board` (the main slot-level defect classifier), `render_overlay`, and `OfflineAOIThread` (a `QThread` wrapping `_aoi_check_board` for the AOI tab).

The slot-classification thresholds in this module were validated on 100-board evaluator runs and mirror those in `aoi_evaluator.py` exactly.

Not a runnable script; imported by `tabs/run_tab.py` and `tabs/aoi_tab.py`.

`threads.py`

Background `QThread` workers:

- `CameraThread` — captures frames at up to 30 fps display / 15 fps inference; selects OS-appropriate backend (`CAP_DSHOW` on Windows, `CAP_AVFOUNDATION` on macOS, `CAP_V4L2` on Linux)
- `InferenceThread` — runs YOLO on frames from the camera
- `ThumbThread` — loads thumbnail images asynchronously
- `TrainingThread` — runs `model.train()` in a background thread
- `AugThread` — augments a dataset in the background
- `AutoLabelThread` — batch-labels a folder of images using YOLO

Not a runnable script; imported by tabs as needed.

`theme.py`

All colour constants, font objects (`_F_MONO_8` through `_F_MONO_12`), and small widget factory functions (`make_card`, `make_sep`, `sec_lbl`, `set_fg`, `_card_style`, `_pen_dash`). Used by every tab and widget module to maintain a consistent visual style.

Not a runnable script; imported throughout the package.

widgets.py

Custom Qt widget library. Contains: `StatCard`, `AnnunciatorBanner`, `FastLog` (high-performance `QPlainTextEdit` with 100 ms batch flush and 500-line cap), `LabelCanvas` (interactive image labeling widget), `VideoWidget`, `ResultStrip`, `ProjectDialog`, `ToastManager`, `SysLogTab`, and `InferTab` (single-image inference viewer, also re-exported via `tabs/infer_tab.py`).

Not a runnable script; imported by tabs.

main_window.py

Application entry point for the modular GUI. Defines `Sidebar` (the left navigation panel with icon buttons), `MainWindow` (the top-level `QMainWindow` with a `QStackedWidget` holding all tabs), and the `main()` function. Sets per-OS thread limits for OpenBLAS/MKL before importing Qt.

Usage:

```
python modular_aoi/ois/main_window.py
```

4.4 `modular_aoi/ois/tabs/`

Each file is a `QWidget` subclass implementing one tab of the GUI. None have `__main__` blocks and none are standalone scripts.

File	Class	Description
`run_tab.py`	`RunTab`	Live camera inspection tab. Manages `CameraThread`, `InferenceThread`, golden reference capture, per-frame PASS/FAIL decision, and the live pixel-diff overlay.
`train_tab.py`	`GoldenTab`	Golden reference and dataset management. Handles image capture to dataset, `AugThread` augmentation, `AutoLabelThread` auto-labelling, and `TrainingThread` model training.
`filters_tab.py`	`LogicTab`	Filter pipeline designer. Displays the filter node list, parameter editors, live preview, and the `AutoCalibrateWorker` controls.
`aoi_tab.py`	`OfflineAOITab`	Offline AOI batch inspection. Runs `OfflineAOIThread` over a folder of images against a loaded golden reference; shows per-image results and exports JSON reports.
`infer_tab.py`	`InferTab`	Compatibility re-export of `InferTab` from `widgets.py`. Single-image and folder inference viewer.
`history_tab.py`	`HistoryTab`	Inspection history viewer backed by `HistoryDB`. Filterable table of past inspection sessions with export.

5 Data & Assets

5.1 `data/Main_dataset/`

The primary labelled training dataset. Contains 23 PCB images with YOLO-format `.txt` annotations and a `data.yaml` class map. This is the starting point for `train/augment_dataset.py` (pass it as `--src`).

```
data/Main_dataset/
```

```
+-- data.yaml          # nc: 25, names: [Connector (P), Resistor (R),
    ...]
+-- images/            # 23 labelled images (.jpg / .png)
\-- labels/            # Matching YOLO .txt annotation files
```

5.2 data/Raw pics/

28 original unlabelled source images — the raw photos before annotation. Includes 5 high-resolution real-world boards captured with a Riyan DSLR (filenames starting `Riyan_20260324_...`). Use these as input to `src/melss_labeler.py` when adding new annotations, or as golden board candidates for `src/generate_defect.py`.

5.3 data/Real_images/TEST/

7 real captured board photos used as an unlabelled held-out validation set. The `labels/` subdirectory is empty — these images are intended to be run through the inspector to verify real-world detection performance, not for training.

```
data/Real_images/TEST/
+-- data.yaml          # Same 25-class definition
+-- images/            # 7 Riyan DSLR boards
\-- labels/            # Empty -- no ground-truth annotations
```

5.4 test_defects/

A pre-generated synthetic defect dataset created by running `src/generate_defect.py` on `original.jpg`. Ready to use immediately with `src/aoi_evaluator.py` or the AOI tab of the GUI.

```
test_defects/
+-- original.jpg             # Golden board used to generate this set
+-- golden_components.json   # YOLO detections on the golden board
+-- manifest.json            # Ground-truth defect map for all 20
   boards
\-- images/
        board_000.png ... board_019.png
```

To run the evaluator against this pre-generated set:

```
python src/aoi_evaluator.py \
    --golden test_defects/original.jpg \
    --model  models/best.pt \
    --boards 20 \
    --defects 7
```

To compare the inspector's output JSON against the ground-truth manifest:

```
python src/generate_defect.py \
    --output-dir test_defects \
    --compare    <path_to_inspector_output.json>
```

6 Typical Workflows

6.1 Workflow 1: Train a new model from scratch

Collect PCB images, label them with the GUI labeler, then run the full training pipeline:

```
# Step 1: Label your images
python src/melss_labeler.py
# Use the GUI: Create Project -> add images -> draw boxes ->
    auto-label

# Step 2: Run all 4 training stages
python train/pipeline.py \
    --src      /path/to/labelled_project \
    --dst-aug  /path/to/my_aug \
    --dst-tile /path/to/my_tiled \
    --model    models/best.pt \
    --name     my_model_v1 \
    --epochs   120 \
    --imgsz    960 \
    --batch    4 \
    --device   0
# Trained weights: <PROJECT_DIR>/my_model_v1/weights/best.pt
# Copy best.pt to models/best.pt for use in inspection scripts
```

6.2 Workflow 2: Generate a synthetic defect dataset and evaluate accuracy

```
# Step 1: Generate 50 synthetic defective boards (up to 7 defects
    each)
python src/generate_defect.py \
    --golden   /path/to/golden_board.jpg \
    --model    models/best.pt \
    --num      50 \
    --defects-per-board 7 \
    --output-dir generated_defects

# Step 2: Run the accuracy evaluator (inject -> detect -> report)
python src/aoi_evaluator.py \
    --golden  /path/to/golden_board.jpg \
    --model   models/best.pt \
    --boards  100 \
    --defects 7 \
    --verbose \
    --debug
# Prints precision, recall, F1, and a defect-type confusion matrix
```

6.3 Workflow 3: Inspect test boards against a golden reference (CLI)

```
# Step 1: Extract golden reference (mode 1)
python src/pcb_aoi.py
# At the prompts:
#   model path   -> models/best.pt
#   output dir   -> /path/to/aoi_output
#   Mode         -> 1  (extract golden only)
#   golden image -> /path/to/golden_board.jpg

# Step 2: Check test boards using saved golden (mode 2)
python src/pcb_aoi.py
# At the prompts:
#   model path      -> models/best.pt
#   output dir      -> /path/to/aoi_output
#   Mode            -> 2  (check test boards)
#   golden JSON     -> /path/to/aoi_output/golden_reference.json
```

```
#   golden image     -> /path/to/aoi_output/golden_raw.jpg
#   test image path -> /path/to/board_001.jpg
#   test image path -> /path/to/board_002.jpg
#   (empty line to finish)
# Results written to /path/to/aoi_output/
```

Alternatively, launch the full GUI for live camera + offline AOI in one application:

```
# Monolithic version
python src/optical_inspection_system.py

# Modular version (identical functionality)
python modular_aoi/ois/main_window.py
```